INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signatur
01	01/07	Introduction & basic HTML	CO1	10	
02	08/07	Design HTML using audio	CO1	10	
03	15/07	Design HTML using video	CO1	10	
04	22/07	Program using JS (arrays)	CO1	10	
05	29/07	Develop dynamic webpage	CO1	10	
06.	05/08	Implement modules in nodejs	CO2	10	
07.	12/08	Mini application using express & nodejs	CO2	10	
08.	19/08	Code tantra (static-webpage)	CO1	9	
09.	2/09	Code tantra (portfolio)	CO1	10	
10.	9/09	Code tantra (simple calculator)	CO1	10	
11.	16/09	Create a functional & class	CO2	10	
12.	23/09	Implement http methods	CO2	10	
13.	30/09	Develop CRUD operations	CO3	10	
~~14.~~					
~~15.~~					

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222. Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	**Semester:**	I
Name of the Student:	T. Tejaswani	**Roll Number:**	A24126552181
Name of the Lab:	FSLab	**Timings:**	9:40-12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
01/07	01	4	4	5	2	15	10	
08/07	02	4	4	5	2	15	10	
22/07	03	4	4	5	2	15	10	
29/07	04	4	4	5	2	15	10	
05/08	05	4	4	5	2	15	10	
12/08	06	4	4	5	1	14	10	
16/09	07	4	4	5	2	15	10	
23/09	08	4	4	5	2	15	10	
30/09	09	4	4	5	2	15	10	

Name of the Faculty: Dr. G. Prasanna Priya

Faculty Signature:

Introduction to HTML and CSS:

HTML (hypertext markup Language) is the standard markup language used to create and structure content on Webpages. It defines the basic structure of webpage using elements and tags such as headings, paragraphs, Links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaning -ful manner. HTML provides structure and semantic organization of a webpage.

CSS (cascading style sheets) is a stylesheet language used to control the presentation & visual appearance of HTML documents. It is used to define properties such as colors, fonts, background, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sized and devices. CSS can be applied using inline, internal or external

- HTML and CSS work together to develop webpages. Where HTML provides the structure and content, While CSS provides styling. The combination of

both forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces

- Difference between MERN stack and Full stack:

MERN stack is a javascript based technology stack used for developing modern web applications. MERN stands for MangoDB, express.js, react.js, node.js, mangoDB is used for database management, express.js handles the backend framework and API's, react.js is used to develop frontend user interface. and node.js provides server side runtime environment.

- Full-stack development refers to development of both frontend and backend of a web applications along with database integration. A Full stack developer work with technologies.

HTML Elements:

1. HTML tags: Define the structure and elements of web page
2. Headings: Used to display titles and heading using <h1> to <h6>
3. paragraphs: Used to display text using <p> tag
4. Links: Used to navigate between webpages using <a> tag.
5. Images: Used to display images using <image> tag
6. Lists: Used to create ordered & unordered lists using <ol>, <ul>,
7. Tables: Used to organize data is rows and columns
8. Forms: Used to collect user input using elements such as text boxes, buttons and checkboxes.
9. Div and span: Used to group and organize webpage content
10. Semantic Elements: Elements such as <header>, <nav>, <selection>, <article>, <footer> provides meaningful Webpage structure.

CSS Components:

01. Selectors - Select HTML elements to apply styles
02. properties - Define the visual characteristics of element
03. Values - specify the setting of css property
04 colors - Define text, background, borders, colors
05. Fonts - Control font family, size, style and weight
06. Background - set background colors and images
07. Borders Define the border style, width and radius
08. Margin Controls space outside an element.
09. Padding Controls the space inside an element
10. Layout Controls arrangement of elements using flexbox, grip and positioning.

- HTML vs HTML5:

HTML	HTML5
1. older version of HTML	1. Latest major version HT
2. Limited multimedia support	2. supports audio and vid
3. Uses external plugins for multimedia	3. No external plugs need
4. Has fewer semantic tags	4. provides semantic tags
5. No <canvas> element	5. supports <canvas>
6. Limited Webstorage	6. supports local storage

- CSS vs CSS3

CSS	CSS3
1. Earlier version of CSS	1. Advanced version of CSS
2. Basic styling features	2. provides advanced styli - g features
3. Limited Animations	3. supports animations and transitions
4. No flexible layout systems	4. supports flexbox and grid

Experiment - 02

Design a static webpage using HTML5 and CSS3

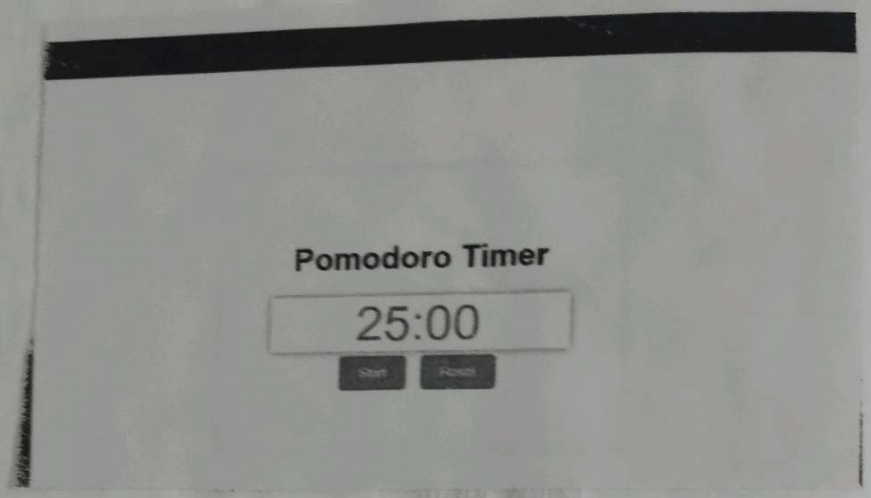
• pomodoro timer - using HTML, CSS

```
<!Doctype html>
<html>
<head>
<title> Pomodoro Timer </title>
<style>
body {
    font-family : Asial ;
    background : #f5f5f5 ;
    text-Align : center ;
    padding : 50px ;
}
timer {
    width : 300px ;
    margin : auto ;
    padding : 30px ;
    back-ground : white ;
    border-radius : 15px ;
    box-shadow : 0 0 10px grey ;
}
h1 {
    color : #e74c3c ;
}
```

```
.time {
    font-size: 50px;
     margin: 20px;
  }
button {
    padding : 10px 20px;
    margin : 5px;
     border : none;
     border-radius : 5px;
     background : #e74c3c;
     color: White;
}
</style>
</head>
 <body>
      <div class = "timer" >
       <h1> Pomodoro timer </h1>
       <div class = "time"> 25:00 </div>
       <button > start </button>
        <button > Reset </button>

    </div>
</body>
</html>
```

output:



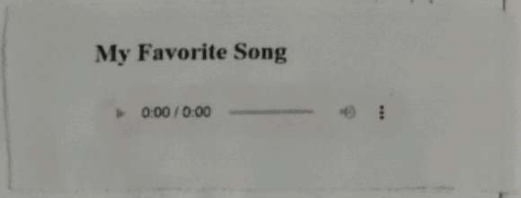
Experiment-01

01. Design an HTML code using audio Tags

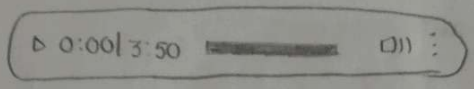
```
<!DOCTYPE html>
<html>
  <head>
    <title> Audio </title>
  </head>
  <body>
    <h2> my favourite song </h2>
    <audio controls>
      <source src = "song.mp3" type = "audio/
                                    mpeg" >
      Your browser doesn't support audio
      tag
    </audio>
  </body>
</html>
```

output:

Displays an audio player with play, pause and volume controls



- My Favourite song



02. Design a HTML code using table tag

code:

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>25</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>30</td>
  </tr>
</table>
```

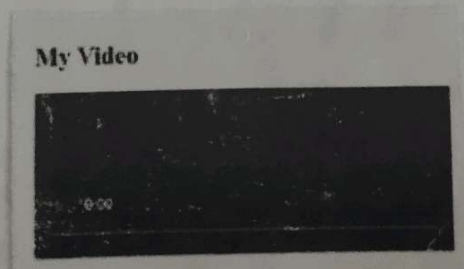
output:

Name	Age
Alice	25
Bob	30

03. Design a HTML code using video tags.

• Code:

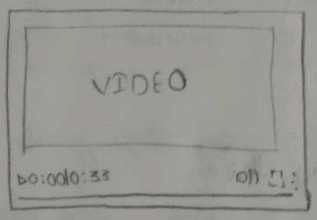
```
<!Doctype html>
<html>
  <head>
    <title> video example </title>
  </head>
  <body>
    <h2> my video </h2>
    <video width = "400" controls>
      <source src = "video.mp4"
              type = "video/mp4">
    </video>
  </body>
</html>
```



output:

Displays a video player with play, pause volume and full screen controls.

- My video



04. Design an HTML code using image tag

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <title> Image </title>
</head>
<body>
   <img src="https://flower.com"
        alt = "FlowerImage">
</body>
</html>
```

output:

FlowerImage

Experiment-03

3. Write a program using Javascript (Arrays, Functions)

Code:

```
Let numbers = [12,13,14,15,16,19,24,34,67,90,99]

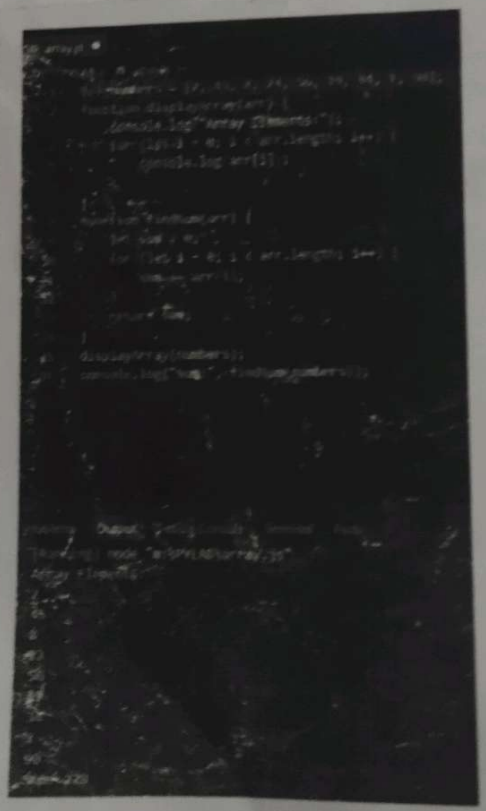
Function display(arr){
    console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}

Function Findsum(arr){
    let sum=0
    for(let i=0; i<arr.length; i++){
        sum+=arr[i]
    }
    return sum;
}
```

Output

12
13
14
15
16
19
24
34
67
90
99

sum : 403



Experiment-04

04. Develop dynamic webpage using java script

program:

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Webpage </title>
    <style>
        body {
            text-align : center;
            font-family : Arial;
            margin-top : 100px;
        }
        button {
            padding : 10px 20px;
            font-size : 16px;
            cursor : pointer;
        }
    </style>
</head>
<body>
    <h1 id = "heading" > Welcome to My Webpag
                                    </h1>
```

```
<p id = "message" > Click the button to change
                    this webpage. </p>

<button onclick = "changePage()" > Click Me
                                  </button>

<script> #

  function changePage() {

  document.getElementById("heading").
      innerHTML = "Welcome to javascript!";

  document.getElementById("message").
      innerHTML = "The webpage has been
changed dynamically.";

  document.body.style.backgroundColor
        = "lightblue";
        }
  </script>

</body>

</html>
```

Output

Welcome to My Webpage

click the button to change this webpage

click Me

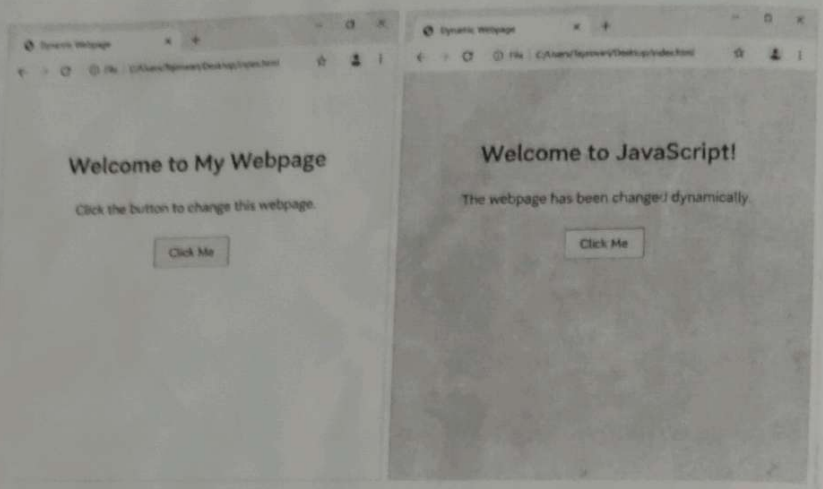
- When the user clicks "click Me", the javascript function changePage() is called, then the heading dynamically changes and background also changes.

After clicking the button:

Welcome to Javascript!

The webpage has been changed dynamically

click Me



Before clicking the button

After clicking the button

Experiment 05

05. Implement Modules in NodeJs

- In Node.js, you can implement modules using two distinct systems: commonJS (CJS), which is traditional default, and ES Modules (ESM), the modern javascript standard

1. The commonJS Method (default):

CommonJS uses module.exports to share code and require() to import it. This system loads modules synchronously

a. Create the module (math.js):

```
Const add = (a,b) => a+b;
const subtract = (a,b) => a-b;

module.exports = {
    add,
    Subtract
};
```

b. use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
```

```
console.log(math.subtract(10,5));
```

output: 15
5

2. The Es modules Method (modern standard):

Es modules use export to share code and import to use it. To make Node.js recognize Es modules, You must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js):

```
export const multiply = (a,b) => a * b
export const divide = (a,b) => a/b

const calculatorName = "supercalc";
export default calculatorName;
```

b. use the module (main.js)

```
import calculatorName, {multiply, divide}
    from './calculator.js';
```

```
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

Output: SuperCalc
8
2

3. Using Built-in core modules:

Node.js also comes equipped with dozens of built-in core modules (like fs for file systems or path for file routing) that don't need installation.

We will load them by module names:

```
// CommonJS style
const fs = require('fs');

// ES modules style
import fs from 'fs';
```

Here are some of the most commonly used ones;

1. fs = file system operations

2. http - HTTP server and client
3. path - File path utilities
4. os - operating system utilities
5. events - Event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. querystring - URL query string handling

Example: Simple HTTP server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end('Hello World!');
}).listen(8080);
```

Output: Hello World!

Experiment 06

06. Develop mini application using express and NodeJS

- Simple mini application using node JS + Express JS

Student Management system

1. Add a student
2. View all students
3. delete a student
4. Uses Express.js routes
5. Simple HTML interface

project structure:

```
student-app/
├── server.js
└── public/
    └── index.html
```

server.js:

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
```

```
app.get("/students", (req, res) => {
    res.json(students);
});

// Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push(student);
    res.json({ message: "student added
                        successfully",
               student: student });
});

// Delete
app.delete("/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  students = students.filter(student =>
    student.id !== id);
res.json({
    message: "student deleted successfully" });
```

```
});
// start server
app.listen(PORT, () => {
  console.log('server running at http://localhost:
    ${PORT}');
});
```

public/index.html:

```
<!DOCTYPE html>
<html>
 <head>
   <title> Student Management </title>
 </head>
 <body>
   <h1> Student Management system </h1>
   <input type = "text" id = "name"
   placeholder = "Enter student name" >
   <input type = "text" id = "course"
   placeholder = "Enter course">
   <button onclick = "addStudent()"> Add student
                                    </button>
   <h2> Student List </h2>
```

```
<ul id="studentList"></ul>

<script>
  function addStudent() {
    let name = document.getElementById("name").
                                        value;
    let course = document.getElementById("course").
                                        value;

    fetch("/students", {
        method: "post",
        headers: {
            "content-Type": "application/json" },
        body: JSON.stringfy({
            name: name,
            course: course
        })
    })
    .then(response => response.json())
    .then(data => {
        alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
    displayStudents();
    });
}
```

```
function displayStudents() {
  fetch("/students").then(response => response.json())
  .then(students => {
    let list = document.getElementById("studentList");
    list.innerHTML = "";
  students.forEach(student => { let li =
  document.createElement("li");
      li.innerHTML = student.id + "-" +
                     student.name + "-" +
                     student.course + "<button
  onclick='deleteStudent(" + student.id + ")'>Delete
  </button>";
    list.appendChild(li);
              });
            });
}

function deleteStudent(id){
    fetch("/students/" + id, { method: "DELETE"})
  .then(response => response.json())
  .then(data => {
        alert(data.message);
        displayStudents();
      });
```

```
}
displayStudents();
</script>
</body>
</html>
```

output:

Student Management system

Enter student name

Enter course

Add student

Student List:

1. Tejaswini - CSM [delete]
2. Sita - CSE [delete]

- The Express.js server is succesfully started at:

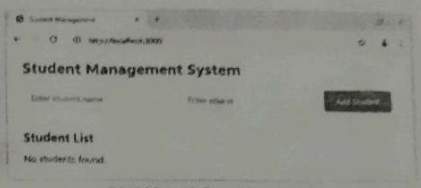
http://localhost:3000

The browser displays the student management system with fields

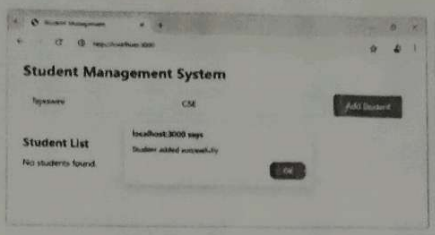
Output:



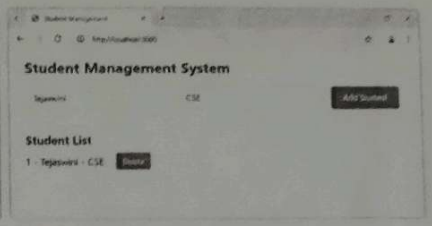
1. Terminal Output (after running `node server.js`)



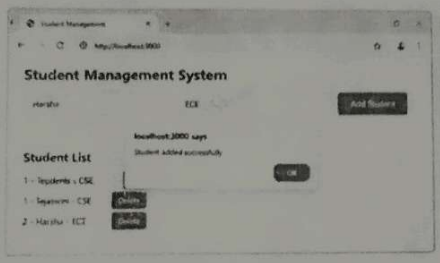
2. Initial Output in Browser (no students)



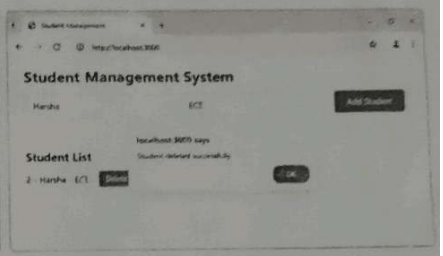
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment - 07

07. Create a Functional component and class component using react.

- Aim: To develop a react application using Functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a web page.

- Functional Component:

A Functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as useState to manage state and update the webpage dynamically

program:

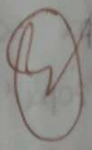
```
import React, {useState} from
'react';

function studentFunctional(){
    const [name, setName] = useState("Tejaswaari");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Teju");
    };
```

```
const changeCourse = () => {
    setCourse ("CSD");
};
const incrementCount = () => {
    setCount(count+1);
};
return (
  <div style = {{
      border : "1px solid blue",
      padding : "20px",
      margin : "20px",
      borderRadius : "10px",
      textAlign : "center"
  }}>
  <h1> StudentDetails </h1>
  <h2> Functional Component </h2>
  <p> <b> Name: </b> {name} </p>
  <p> <b> course : </b> {course} </p>
  <p> <b> Button clicks: </b> {count} </p>
  <button onClick = {changeName} >
      changeName </button>
  <button onClick = {changeCourse} change
      course </button>
```

```
<button onClick = {incrementCount}> Increment
        </button>
    </div>
);
```

output: When the user clicks the buttons, the name, course and count change dynamically without reloading the webpage.



• class component:

A class component is an ES6 JavaScript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

program:

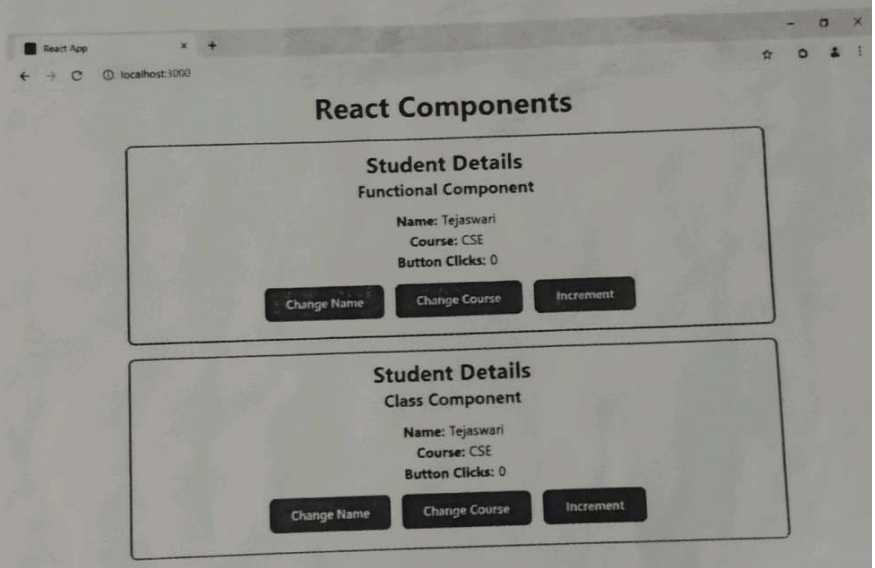
```
import React, {Component} from
'react';
class studentClass extends Component {
    constructor(props) {
        super(props);
        this.state = {
            name : "Tejaswari",
            course : "Computer Science",
            count : 0
        };
    }
    changeName = () => {
        this.setState({
            name : "Teju"
        });
    };
    changeCourse = () => {
        this.setState({
            course : "CSM"
        });
    };
```

```
incrementCount = () => {
    this.setState((prevState) => ({
        count: prevState.count + 1
    }));
};

render() {
  return (
    <div style={{
        border: "8px solid green",
        padding: "30px",
        margin: "30px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
        <h1>Student Details</h1>
        <h2>Class Component</h2>
      <p><b>Name:</b> {this.state.name}</p>
      <p><b>Course:</b> {this.state.course}</p>
      <p><b>Button clicks:</b> {this.state.count}</p>
    <button onClick={this.changeName}>
            changeName</button>
    <button onClick={this.changeCourse}>
            Change Course</button>
```

```
<button onClick={this.incrementCount}>Increment
</button>
</div>
```

output:



Experiment : 08

08. Implement http methods using JSON server

- Aim : To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, update and Delete) operations on student records

Theory : JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record from the server

a. Create a JSON server:

step 1 : Initialize the project

open the vs code terminal and execute the following commands:

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

step 2: Create db.json

Create a file named db.json and add the following student data.

```
{
  "students": [
    { "id": "1",
      "name": "Tejaswari",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "chaitsika",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "3",
      "name": "saavani",
      "course": "IT",
    } "marks": 88
```

step 3: start the JSON server

For the current JSON server version, execute:

npx json-server db.json

The server start at:

http://localhost:3000

The student records can be accessed using:

http://localhost:3000/students

Output:

http://localhost:3000

http://localhost:3000/students

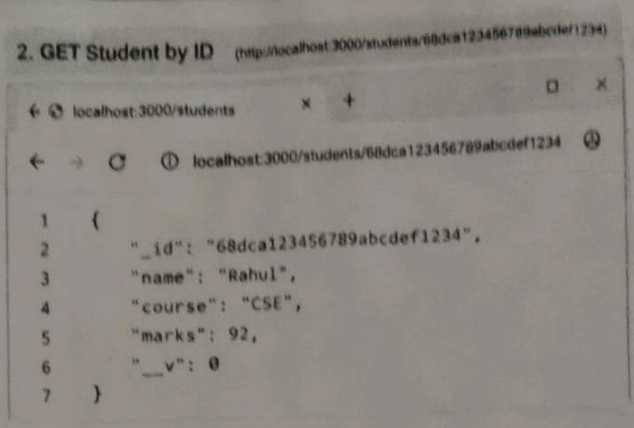
3. Implementation of HTTP methods

a. GET Method: The GET method is used to retrieve student records from JSON server

Code:

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

output:



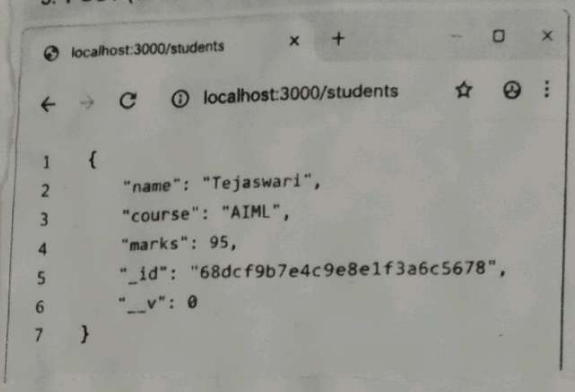
B. POST Method: The POST method is used to insert a new student record into the server

code:
```
fetch ('http://localhost:3000/students', {
    method: 'POST',
    headers: {
      'content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Rahul',
        course: 'CSE',
        marks: 92
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
```

```
• catch (error => {
    console.log(error);
});
```

3. POST (Create Student) – Using Postman

output:



c. PUT Method: The PUT method is used to replace an existing student record with updated data.

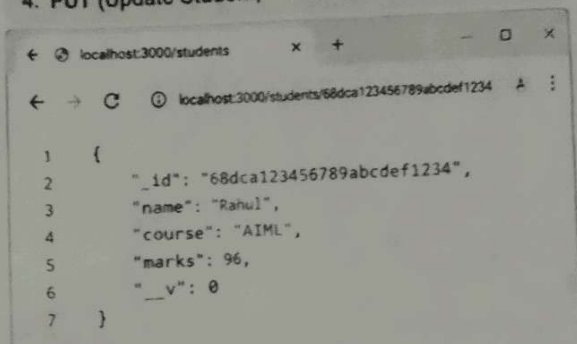
code:

```
fetch('http://localhost:3000/students/1', {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Tejaswari',
        course: 'AIML',
        marks: 95
    })
})
• then(response => response.json())
```

```
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
```

output

4. PUT (Update Student) – Using Postman



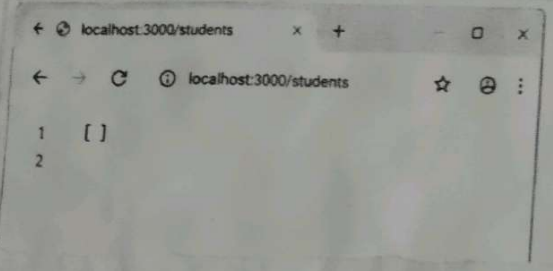
D. PATCH Method: The PATCH method is used to update only selected fields of an existing student record.

code:

```
fetch('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      marks: 96
    })
})
.then(response => response.json())
.then(data => {
```

```
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

output:



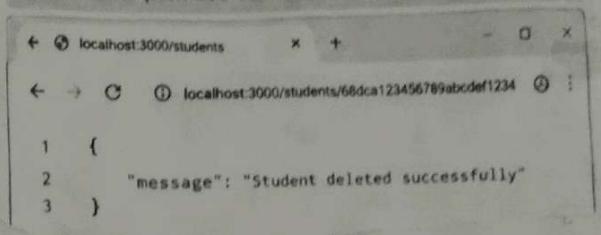
E. DELETE Method: The DELETE method is used to remove an existing student record from the server

Code:

```
fetch('http://localhost:3000/students/3', {
    method: 'DELETE'
})
.then(response => {
  if (response.ok) {
    console.log("student deleted
                successfully");
  }
})
.catch(error => {
    console.log(error);
});
```

output:

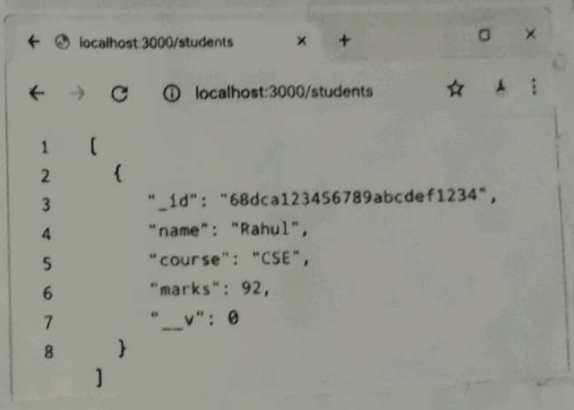
5. DELETE (Remove Student) – Using Postman



- After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be:

output:

1. GET All Students (http://localhost:3000/students)



Thus, HTTP methods GET, POST, PUT, PATCH and DELET
were successfully implemented using JSON Server. Th
CRUD operations were performed on student records
the data was retrieved, inserted, updated and
deleted using HTTP requests.

Experiment-9

09. Develop CRUD operations using MongoDB.

Aim: To develop a web application using Node.js, Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and project setup:

```
mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

2. Program: Server.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/college')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
// create student
const studentSchema = new
mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    course: {
        type: String,
        required: true
    },
    marks: {
        type: Number,
        required: true
```

```
});

// create student model
const student =
  mongoose.model('student', studentSchema);

// CREATE : Add
app.post('/students', async (req, res) => {
  try {
    const student = new Student(req.body);
    const savedStudent = await student
                          .save();
    res.status(201).json(savedStudent);
  } catch(error) {
    res.status(400).json({ message:
                  error.message });
  }
});

// READ : Get all students
app.get('/students', async (req, res) => {
  try {
    const students = await
```

```
    Student.find();
    res.status(500).json({message: error.message});
  }
});

// READ: Get student by ID
app.get('/students/:id', async(req,res)=>{
  try{
    const student = await
    Student.findById(req.params.id);
    if(!student){
      return res.status(404).json({
        message: "student not found"
      });
    }
    res.status(200).json(student);
  }catch(error){
    res.status(400).json({message:
    error.message});
  }
});

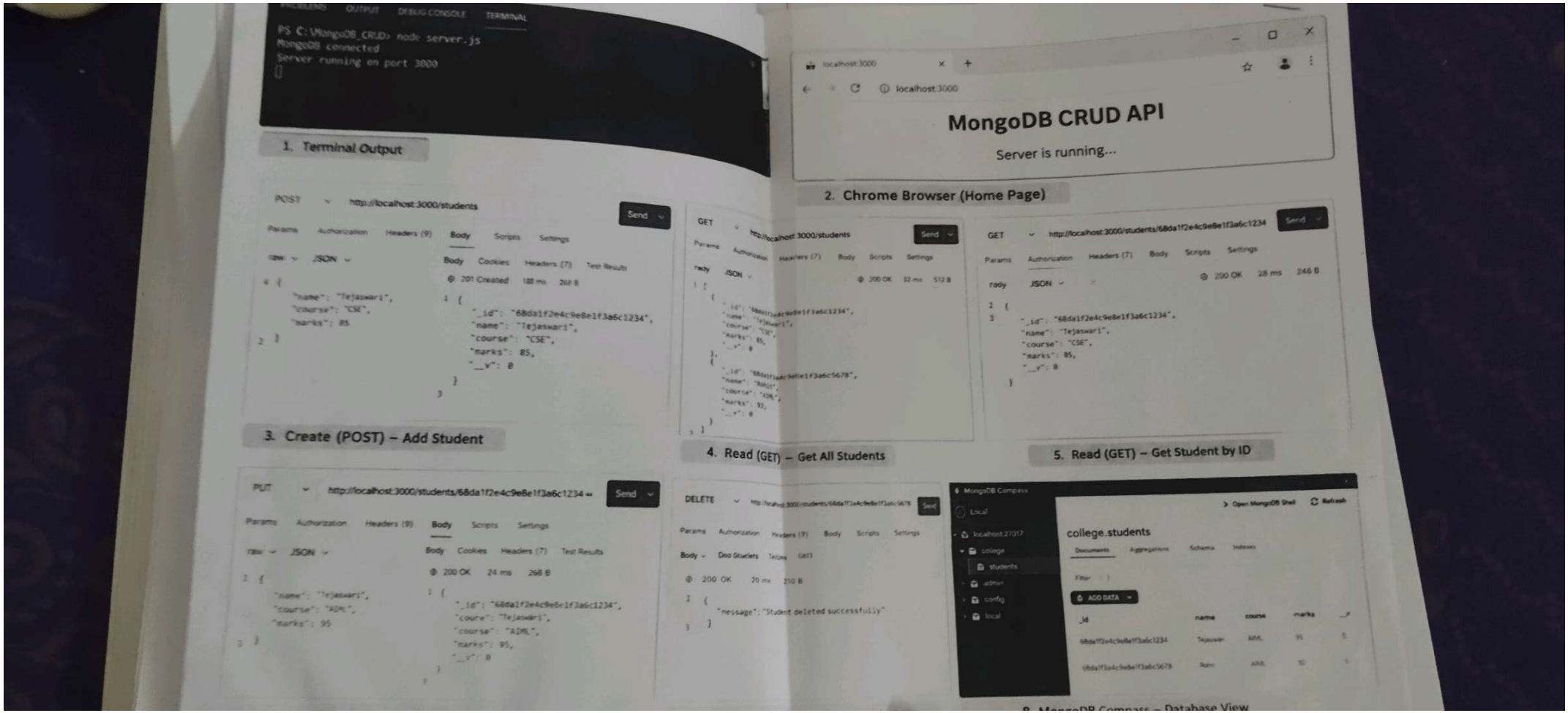
// UPDATE
app.put('/students/:id', async(req, res)=>{
  try{
```

```
    const student = await
Student.findByIdAndUpdate(
        req.params.id,
        req.body,
        {new: true, runValidators: true}
    );
    if (!student) {
        return res.status(404).json({
            message: "student not found"
        });
    }
    res.status(200).json(student);
} catch(error) {
    res.status(400).json({message:
        error.message});
}
});

//DELETE
app.delete('/students/:id', async(req,res)=>{
    try{
        const student = await student.
            findByIdAndDelete(,
```

```
        req.params.id
    );
    if (!student) {
        return res.status(404).json({
            message: "student not found"
        });
    }
    res.status(200).json({
        message: "student deleted
                  successfully"
    });
} catch (error) {
    res.status(400).json({message:
        error.message});
}
});
```

Outputs:


PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
1. Terminal Output
localhost:3000
MongoDB CRUD API
Server is running...
2. Chrome Browser (Home Page)
POST http://localhost:3000/students
Send
3. Create (POST) – Add Student
GET http://localhost:3000/students
4. Read (GET) – Get All Students
GET http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234
5. Read (GET) – Get Student by ID
PUT http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234
DELETE
MongoDB Compass
college.students